

Universidad de Costa Rica
Facultad de Ingeniería
Escuela de Ingeniería Eléctrica
IE0435 Inteligencia Artificial aplicada a la Ingeniería Eléctrica
I ciclo 2026

Proyecto 1 - Entregable 1
Clasificación de “contaminaciones” en una línea de
producción simulada

Jacob González Gámez B83417
Grupo 01

Profesor: Marvin Coto Jimenez

28 de abril del 2026

Índice

1. Contexto	1
2. Objetivo General	2
3. Datos: recolección colaborativa de todo el grupo	3
4. Preprocesamiento	4
4.1. Escogencia del umbral de binarización	4
5. Conversión de matrices a datos csv etiquetados	6
6. Modelos de clasificación	7
6.1. Conjunto de datos consolidado	7
6.2. Modelos evaluados y justificación	7
6.3. Búsqueda de hiperparámetros	7
6.4. Métricas de evaluación	7
6.5. Elección del mejor modelo y generalización	8
6.6. Exportación del modelo	8
7. Anexos	9
Apéndice A. Código Python para el preprocesamiento	9
Apéndice B. Código Python para selección del umbral de binarización	10
Apéndice C. Código Python para la creación de los vectores con etiquetado en formato csv	13
Apéndice D. Código Python para la clasificación de modelos	14
Apéndice E. Código Python para la exportación del modelo	20

1. Contexto

En procesos de manufactura, la inspección visual busca detectar y cuantificar defectos o anomalías para asegurar la calidad y reducir error humano. En investigación aplicada moderna, esto se apoya en aprendizaje automático, especialmente de imágenes, con especial cuidado de iluminación, adquisición y evaluación

2. Objetivo General

Construir un sistema completo, que incluye datos, modelos, evaluación y exportación del modelo, para cumplir con lo siguiente: Detectar elementos anómalos (contaminaciones) en imágenes, usando aprendizaje automático clásico.

3. Datos: recolección colaborativa de todo el grupo

Se simulará la línea de producción con una hoja blanca, a la cual pueden caer granos de arroz, los cuales son contaminaciones indeseadas en una línea de producción. También pueden caer otro tipo de objetos más grandes, que son además aros o clips. Solamente las imágenes donde haya presencia de granos de arroz se consideran ejemplos positivos. Cualquier otro objeto, o ninguno, será un ejemplo negativo. Para generar el conjunto de datos, cada estudiante aporta 30 fotos, 15 que representan contaminaciones positivas y 15 negativas.

En el siguiente enlace se encuentra un recopilado de las imágenes capturadas para propósitos del proyecto:

<https://1drv.ms/f/c/833cd54d0378805a/IgAXdsgYtR40RKRhIGBQca14Acaw00QffatmPnnB6N2aFJc?e=Rmagvb>

En la figura 1 se puede observar una imagen de ejemplo, que se utiliza para el proyecto y que muestra contaminación positiva.



Figura 1: Ejemplo de imagen con contaminación positiva

4. Preprocesamiento

Antes de implementar la clasificación de las imágenes primero se debe hacer un preprocesamiento de estas, de manera que tengan un tamaño standard y se binaricen. Esto es importante porque en este formato las imágenes son más fáciles de procesar por los algoritmos de clasificación.

La primera parte de este preprocesamiento consiste en transformar las imágenes a un tamaño de 128 x 128 píxeles. La segunda parte es la conversión a formato de unos y ceros, es decir la binarización. En la sección de Anexos [A](#), se puede observar el código utilizado para esta parte, realizado con ayuda de Claude Code en VS Code. Este script se encarga del procesamiento inicial de las imágenes. Recorre recursivamente la carpeta **Images** y sus subcarpetas, identificando todos los archivos de imagen compatibles. Para cada imagen, realiza los siguientes pasos: primero la convierte a escala de grises, colapsando los tres canales de color (RGB) en un único canal de intensidad con valores entre 0 y 255; luego la redimensiona a 128×128 píxeles utilizando el método de remuestreo Lanczos [\[1\]](#), que preserva los detalles de los bordes al reducir el tamaño. Finalmente, aplica un umbral de binarización: los píxeles con intensidad mayor al umbral definido se asignan el valor 1 (fondo blanco) y los demás el valor 0 (objeto presente, como un grano de arroz). Todas las matrices binarias resultantes se almacenan en un archivo comprimido **.npz** para su uso posterior.

En la figura [2](#), se puede observar una representación de blancos y negros de una imagen preprocesada de forma completa. En este caso el umbral de binarización es de 128 en la escala de grises. Sin embargo, como se verá más adelante, se tuvo que cambiar el valor para uno más apropiado que representara de mejor forma todas las imágenes recopiladas.

Binary (threshold=128)
white(1): 16063 object(0): 321



Figura 2: Ejemplo de imagen con preprocesamiento

4.1. Escogencia del umbral de binarización

Para la escogencia del umbral de binarización de las imágenes se procedió a realizar un script con ayuda de Claude Code (ver Anexo [B](#)), que permitiera observar de manera práctica las imágenes una vez que fueran procesadas y que se viera el cambio en estas al cambiar dinámicamente el valor del umbral. Al ejecutar el script, despliega una ventana con tres paneles: el primero muestra la imagen en escala de grises, el segundo presenta un mapa de intensidades con una barra de color como referencia numérica, y el tercero muestra la imagen binaria resul-

tante según el umbral actual. Un control deslizante permite modificar el umbral en tiempo real, actualizando de inmediato la imagen binaria y el conteo de píxeles blancos y negros. Además, dos botones de navegación permiten moverse entre todas las imágenes disponibles sin cerrar la ventana. Esto facilita encontrar el valor de umbral más adecuado para el conjunto de imágenes antes de ajustarlo en `image_processor.py`.

En la figura 3 se puede observar esta ventana de ajuste, en este caso se tiene un valor de 128 para el umbral.

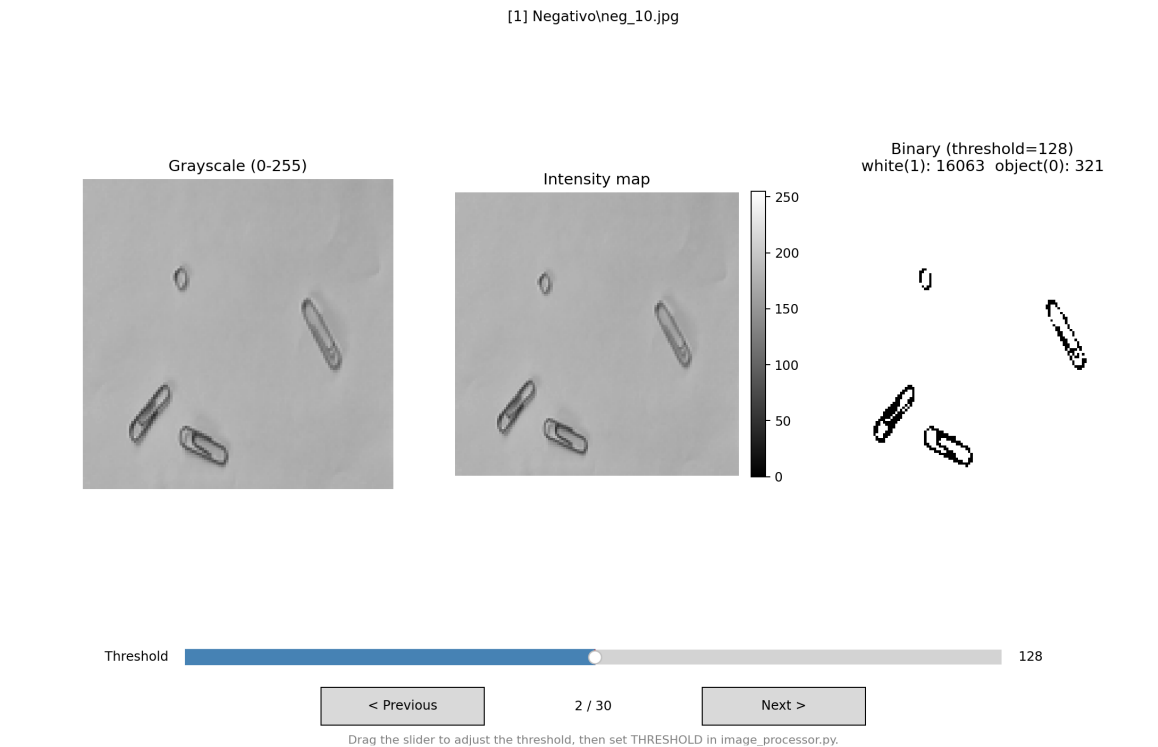


Figura 3: Ventana para la determinación del umbral de binarización

Probando diferentes valores de umbral y viendo el resultado de binarización en cada una de las imágenes originales, se observó que ningún valor permitía tener una representación ideal para la binarización, posiblemente por el poco contraste entre el arroz y los demás elementos. Por lo que, algunas de las imágenes originales fueron retocadas para mejorar este contraste.

Luego se volvieron a probar diferentes valores de umbral y finalmente se optó por el valor 156. En la figura 4 se tiene el resultado de binarización de una de las imágenes.

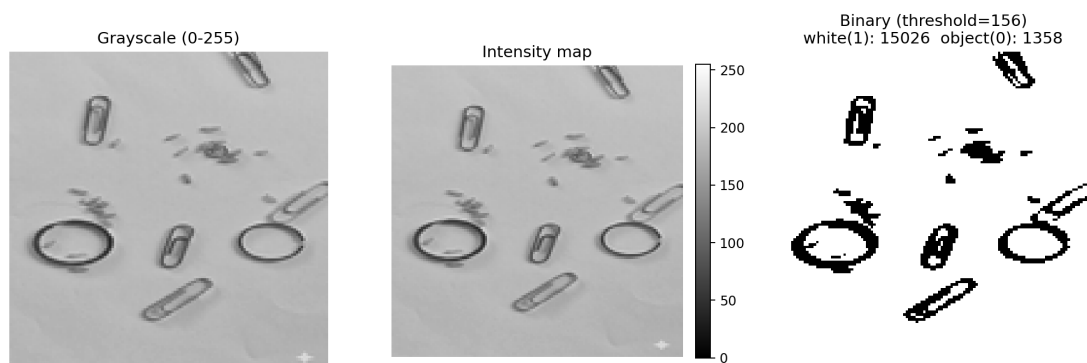


Figura 4: Ejemplo de imagen binarizada con valor de umbral de 156

5. Conversión de matrices a datos csv etiquetados

Cada imagen ya procesada es una cuadrícula de 128×128 celdas, donde cada celda contiene un 1 (fondo blanco) o un 0 (grano de arroz). Para convertirla en un dato que un algoritmo de Machine Learning pueda usar, se necesita transformar esa cuadrícula en una sola fila de números.

1. **Aplanado** — La cuadrícula de 128×128 se “desenrolla” fila por fila, produciendo una lista de 16 384 valores ($128 \times 128 = 16\,384$).
2. **Etiquetado** — Al final de esa lista se agrega un número que indica si la imagen pertenece a la carpeta **Positivo** (contiene arroz $\rightarrow 1$) o a la carpeta **Negativo** (no contiene arroz $\rightarrow 0$).
3. **Exportación** — Cada imagen queda como una fila en el archivo CSV. La primera línea del archivo son los encabezados: `pixel_0, pixel_1, ..., pixel_16383, label`.

El resultado es una tabla donde cada fila representa una imagen completa y la última columna indica si esa imagen tiene o no contaminación por arroz. Esta estructura es la entrada estándar para entrenar modelos de clasificación.

En el Anexo C se puede observar el código utilizado para exportación de los vectores a formato csv. En el siguiente enlace se encuentran los vectores: <https://1drv.ms/f/c/833cd54d0378805a/IgBP1-VT6rJ-RIAJ5QG0PtLJAeIJPjtafF0iSp7ZggM0slE?e=idHFBa>

6. Modelos de clasificación

6.1. Conjunto de datos consolidado

Para el entrenamiento de los modelos se consolidaron los archivos CSV aportados por todos los integrantes del grupo, almacenados en la carpeta `A11_csv`. Tras eliminar filas duplicadas, el conjunto final quedó conformado por 357 muestras balanceadas: 177 ejemplos negativos (sin arroz) y 180 positivos (con arroz). Cada muestra es un vector de 16 384 características binarias más una etiqueta de clase. La partición utilizada fue 80 % para entrenamiento (285 muestras) y 20 % para prueba (72 muestras), con estratificación para preservar la proporción de clases.

6.2. Modelos evaluados y justificación

Se entrenaron cinco modelos de clasificación clásica: Árbol de Decisión, Random Forest, Naive Bayes Bernoulli, K-Nearest Neighbors (KNN) y Support Vector Machine (SVM lineal). La elección de estos modelos responde a su complementariedad: el Árbol de Decisión es interpretable y sirve como línea base; Random Forest extiende esta idea combinando múltiples árboles para reducir el sobreajuste; Naive Bayes Bernoulli es teóricamente adecuado para datos binarios; KNN clasifica por similitud geométrica; y SVM lineal es reconocido por su eficacia en espacios de alta dimensionalidad.

Se eligió **Naive Bayes Bernoulli** sobre Gaussian Naive Bayes porque los datos de entrada son estrictamente binarios (0 o 1), y el modelo Bernoulli está diseñado precisamente para este tipo de variables, mientras que el Gaussiano asume una distribución continua de los datos.

6.3. Búsqueda de hiperparámetros

Para cada modelo se realizó una búsqueda exhaustiva de hiperparámetros mediante *Grid Search* con validación cruzada estratificada de 5 particiones (*5-fold cross-validation*), optimizando la métrica F1. Los rangos evaluados se resumen en la tabla 1.

Tabla 1: Hiperparámetros evaluados por modelo

Modelo	Hiperparámetros evaluados
Árbol de Decisión	$\text{criterion} \in \{\text{gini}, \text{entropy}\}$, $\text{max_depth} \in \{3, 5, 10, 20, \text{None}\}$, $\text{min_samples_split} \in \{2, 5, 10\}$
Random Forest	$\text{n_estimators} \in \{50, 100, 200\}$, $\text{max_depth} \in \{5, 10, 20, \text{None}\}$, $\text{criterion} \in \{\text{gini}, \text{entropy}\}$
Naive Bayes	$\alpha \in \{0.001, 0.01, 0.1, 0.5, 1.0, 2.0, 5.0\}$
KNN	$\text{n_neighbors} \in \{3, 5, 7, 11, 15\}$, $\text{weights} \in \{\text{uniform}, \text{distance}\}$, $\text{metric} \in \{\text{euclidean}, \text{manhattan}\}$
SVM	$C \in \{0.001, 0.01, 0.1, 1.0, 10.0\}$, $\text{loss} \in \{\text{hinge}, \text{squared_hinge}\}$

6.4. Métricas de evaluación

Para evaluar los modelos se utilizaron cuatro métricas estándar de clasificación binaria:

- **Exactitud (Accuracy):** proporción de predicciones correctas sobre el total de muestras.
- **Precisión (Precision):** de todas las muestras clasificadas como positivas, qué fracción realmente lo era. Indica qué tan confiables son las detecciones positivas.

- **Exhaustividad (Recall):** de todas las muestras realmente positivas, qué fracción fue detectada. Indica la capacidad del modelo de no pasar por alto contaminaciones.
- **F1:** media armónica entre precisión y recall. Es la métrica principal de selección, ya que penaliza modelos que sacrifican una métrica por la otra.

En el contexto de inspección de calidad, el recall tiene especial importancia: es preferible generar una falsa alarma (clasificar como contaminado algo limpio) que dejar pasar una contaminación real.

Los resultados sobre el conjunto de prueba se resumen en la tabla 2.

Tabla 2: Resultados de los modelos en el conjunto de prueba (72 muestras)

Modelo	Accuracy	Precision	Recall	F1 prueba	F1 CV
Random Forest	0.736	0.667	0.944	0.782	0.742
Árbol de Decisión	0.639	0.596	0.861	0.705	0.686
Naive Bayes	0.625	0.600	0.750	0.667	0.490
SVM (Lineal)	0.583	0.560	0.778	0.651	0.651
KNN	0.569	0.632	0.333	0.436	0.485

6.5. Elección del mejor modelo y generalización

El modelo seleccionado es **Random Forest** con los hiperparámetros: `criterion = entropy`, `max_depth = 10` y `n_estimators = 200`. Es el que obtiene el mayor F1 tanto en prueba (0.782) como en validación cruzada (0.742), lo que indica que no está sobreajustado y generaliza bien a datos no vistos.

Su matriz de confusión se muestra en la tabla 3.

Tabla 3: Matriz de confusión — Random Forest (conjunto de prueba)

	Predicho Negativo	Predicho Positivo
Real Negativo	19	17
Real Positivo	2	34

El modelo detectó 34 de 36 imágenes con arroz ($\text{recall} = 94.4\%$), dejando pasar únicamente 2 contaminaciones. El F1 de validación cruzada (0.742) es consistente con el F1 de prueba (0.782), lo que confirma que el modelo generaliza de forma estable.

6.6. Exportación del modelo

Una vez identificados los mejores hiperparámetros, el modelo fue reentrenado sobre el conjunto completo de 357 muestras para maximizar la información disponible, y exportado en formato `.joblib` usando la biblioteca `joblib` de Python. El archivo resultante es:

B83417_Jacob_Gonzalez.joblib

El código utilizado para el entrenamiento con todos los datos y la exportación del modelo se encuentra en el Anexo D, y el script de exportación en el Anexo E.

7. Anexos

A. Código Python para el preprocesamiento

```
1 import os
2 import numpy as np
3 from PIL import Image
4
5 IMAGES_DIR = "Images"
6 TARGET_SIZE = (128, 128)
7 # Pixels brighter than this threshold are considered background (white
8 # Pixels at or below are considered object/rice (dark
9 THRESHOLD = 150
10
11
12 def process_image(image_path: str) -> np.ndarray:
13     """Load an image, convert to grayscale 128x128, and binarize to
14     0/1 array."""
15     with Image.open(image_path) as img:
16         img = img.convert("L") # grayscale
17         img = img.resize(TARGET_SIZE, Image.LANCZOS)
18         pixels = np.array(img, dtype=np.uint8)
19
20     # 1 = white background, 0 = object (rice)
21     binary = np.where(pixels > THRESHOLD, 1, 0).astype(np.uint8)
22     return binary
23
24 def process_folder(folder_path: str) -> dict[str, np.ndarray]:
25     """Recursively process all images in folder_path and return a
26     label arrays dict."""
27     results = {}
28     supported = {".jpg", ".jpeg", ".png", ".bmp", ".tiff", ".webp"}
29
30     for root, _, files in os.walk(folder_path):
31         for filename in files:
32             if os.path.splitext(filename)[1].lower() not in supported:
33                 continue
34
35             file_path = os.path.join(root, filename)
36             # Use path relative to IMAGES_DIR as the key
37             rel_key = os.path.relpath(file_path, folder_path)
38
39             try:
40                 binary_array = process_image(file_path)
41                 results[rel_key] = binary_array
42             except Exception as exc:
43                 print(f" [SKIP] {rel_key}: {exc}")
44
45     return results
46
```

```

47 def main():
48     if not os.path.isdir(IMAGES_DIR):
49         raise FileNotFoundError(f"Folder '{IMAGES_DIR}' not found in
           the current directory.")
50
51     print(f"Processing images in '{IMAGES_DIR}' -> grayscale 128x128
           -> binary array\n")
52     images = process_folder(IMAGES_DIR)
53
54     for rel_path, arr in images.items():
55         white_pixels = int(np.sum(arr == 1))
56         object_pixels = int(np.sum(arr == 0))
57         print(f"{rel_path}")
58         print(f"    shape : {arr.shape} | white(1): {white_pixels}
           object(0): {object_pixels}")
59         print(f"    array preview (first row):\n {arr[0]}\n")
60
61     print(f"Total images processed: {len(images)}")
62
63     # Save all arrays to a single .npz file for later use
64     output_file = "binary_images.npz"
65     np.savez_compressed(output_file, **{k.replace(os.sep, "_"): v for
           k, v in images.items()})
66     print(f"\nAll binary arrays saved to '{output_file}'")
67
68
69 if __name__ == "__main__":
70     main()

```

B. Código Python para selección del umbral de binarización

```

1  import os
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import matplotlib.widgets as widgets
5  from PIL import Image
6
7  IMAGES_DIR = "Images"
8  TARGET_SIZE = (128, 128)
9  SUPPORTED = {".jpg", ".jpeg", ".png", ".bmp", ".tiff", ".webp"}
10
11
12 def collect_images(folder: str) -> list[str]:
13     paths = []
14     for root, _, files in os.walk(folder):
15         for f in sorted(files):
16             if os.path.splitext(f)[1].lower() in SUPPORTED:
17                 paths.append(os.path.join(root, f))
18     return sorted(paths)
19

```

```

20
21 def load_grayscale(path: str) -> np.ndarray:
22     with Image.open(path) as img:
23         img = img.convert("L").resize(TARGET_SIZE, Image.LANCZOS)
24         return np.array(img, dtype=np.uint8)
25
26
27 def binarize(gray: np.ndarray, threshold: int) -> np.ndarray:
28     return np.where(gray > threshold, 1, 0).astype(np.uint8)
29
30
31 def main():
32     paths = collect_images(IMAGES_DIR)
33     if not paths:
34         print("No images found in", IMAGES_DIR)
35         return
36
37     print(f"Found {len(paths)} images.")
38     for i, p in enumerate(paths):
39         print(f"    [{i}] {os.path.relpath(p, IMAGES_DIR)}")
40
41     choice = input("\nEnter image number to start from (or press Enter
42                   for 0): ").strip()
43     start = int(choice) if choice.isdigit() and int(choice) < len(
44         paths) else 0
45
46     # Mutable state shared across callbacks
47     state = {"index": start}
48
49     # --- Build figure ---
50     fig, axes = plt.subplots(1, 3, figsize=(14, 5))
51     fig.subplots_adjust(bottom=0.22)
52
53     ax_orig, ax_gray, ax_bin = axes
54
55     ax_orig.axis("off")
56     ax_gray.axis("off")
57     ax_bin.axis("off")
58
59     img_orig = ax_orig.imshow(np.zeros(TARGET_SIZE), cmap="gray", vmin
60                               =0, vmax=255)
61     ax_orig.set_title("Grayscale (0-255)")
62
63     img_gray = ax_gray.imshow(np.zeros(TARGET_SIZE), cmap="gray", vmin
64                                 =0, vmax=255)
65     ax_gray.set_title("Intensity map")
66     plt.colorbar(img_gray, ax=ax_gray, fraction=0.046, pad=0.04)
67
68     img_bin = ax_bin.imshow(np.zeros(TARGET_SIZE), cmap="gray", vmin
69                               =0, vmax=1)
70     title_bin = ax_bin.set_title("")
71
72     suptitle = fig.suptitle("", fontsize=11)

```

```

68
69 # --- Slider ---
70 ax_slider = fig.add_axes([0.2, 0.1, 0.6, 0.04])
71 slider = widgets.Slider(
72     ax_slider, "Threshold", valmin=0, valmax=255,
73     valinit=128, valstep=1, color="steelblue"
74 )
75
76 # --- Navigation buttons ---
77 ax_prev = fig.add_axes([0.3, 0.03, 0.12, 0.05])
78 ax_next = fig.add_axes([0.58, 0.03, 0.12, 0.05])
79 btn_prev = widgets.Button(ax_prev, "< Previous")
80 btn_next = widgets.Button(ax_next, "Next >")
81
82 # Counter label between buttons
83 ax_counter = fig.add_axes([0.43, 0.03, 0.14, 0.05])
84 ax_counter.axis("off")
85 counter_text = ax_counter.text(
86     0.5, 0.5, "", ha="center", va="center", fontsize=10, transform
87     =ax_counter.transAxes
88 )
89
90 fig.text(
91     0.5, 0.005,
92     "Drag the slider to adjust the threshold, then set THRESHOLD
93     in image_processor.py.",
94     ha="center", fontsize=9, color="gray"
95 )
96
97 def refresh():
98     idx = state["index"]
99     gray = load_grayscale(paths[idx])
100     t = int(slider.val)
101     binary = binarize(gray, t)
102
103     img_orig.set_data(gray)
104     img_gray.set_data(gray)
105     img_bin.set_data(binary)
106
107     label = os.path.relpath(paths[idx], IMAGES_DIR)
108     suptitle.set_text(f"[{idx}] {label}")
109     title_bin.set_text(
110         f"Binary (threshold={t})\n"
111         f"white(1): {int(np.sum(binary==1))}   object(0): {int(np.
112             sum(binary==0))}"
113     )
114     counter_text.set_text(f"{idx + 1} / {len(paths)}")
115     fig.canvas.draw_idle()
116
117 def on_slider(_val):
118     refresh()
119
120 def on_next(_event):

```

```

118         state["index"] = (state["index"] + 1) % len(paths)
119         refresh()
120
121     def on_prev(_event):
122         state["index"] = (state["index"] - 1) % len(paths)
123         refresh()
124
125     slider.on_changed(on_slider)
126     btn_next.on_clicked(on_next)
127     btn_prev.on_clicked(on_prev)
128
129     refresh()
130     plt.show()
131
132
133 if __name__ == "__main__":
134     main()

```

C. Código Python para la creación de los vectores con etiquetado en formato csv

```

1  import numpy as np
2  import csv
3  import os
4
5  NPZ_FILE = "binary_images.npz"
6  OUTPUT_CSV = "dataset.csv"
7  POSITIVE_FOLDER = "Positivo"    # label = 1
8  TARGET_SIZE = (128, 128)
9  N_PIXELS = TARGET_SIZE[0] * TARGET_SIZE[1]    # 16384
10
11
12 def get_label(key: str) -> int:
13     """Return 1 if image comes from the Positivo folder, 0 otherwise."""
14     return 1 if key.startswith(POSITIVE_FOLDER) else 0
15
16
17 def main():
18     if not os.path.isfile(NPZ_FILE):
19         print(f"'{NPZ_FILE}' not found. Run image_processor.py first.")
20         return
21
22     data = np.load(NPZ_FILE)
23     keys = sorted(data.files)
24
25     # Column headers: pixel_0, pixel_1, ..., pixel_16383, label
26     headers = [f"pixel_{i}" for i in range(N_PIXELS)] + ["label"]
27
28     positives = sum(1 for k in keys if get_label(k) == 1)

```

```

29     negatives = len(keys) - positives
30     print(f"Images found: {len(keys)} (Positivo=1: {positives},
        Negativo=0: {negatives})")
31     print(f"Row vector size: {N_PIXELS} pixels + 1 label = {N_PIXELS +
        1} columns")
32     print(f"Writing '{OUTPUT_CSV}'...")
33
34     with open(OUTPUT_CSV, "w", newline="") as f:
35         writer = csv.writer(f)
36         writer.writerow(headers)
37
38         for key in keys:
39             arr = data[key]
40             row_vector = arr.flatten().tolist()
41             label = get_label(key)
42             writer.writerow(row_vector + [label])
43
44     size_kb = os.path.getsize(OUTPUT_CSV) / 1024
45     print(f"Done. '{OUTPUT_CSV}' {len(keys)} rows x {N_PIXELS + 1}
        columns ({size_kb:.1f} KB)")
46
47
48 if __name__ == "__main__":
49     main()

```

D. Código Python para la clasificación de modelos

```

1  import os
2  import glob
3  import warnings
4  import numpy as np
5  import pandas as pd
6  from sklearn.model_selection import train_test_split, GridSearchCV,
    StratifiedKFold
7  from sklearn.feature_selection import VarianceThreshold
8  from sklearn.tree import DecisionTreeClassifier
9  from sklearn.ensemble import RandomForestClassifier
10 from sklearn.naive_bayes import BernoulliNB
11 from sklearn.neighbors import KNeighborsClassifier
12 from sklearn.svm import LinearSVC
13 from sklearn.metrics import (
14     accuracy_score, precision_score, recall_score,
15     f1_score, classification_report, confusion_matrix
16 )
17 from sklearn.pipeline import Pipeline
18 from sklearn.preprocessing import StandardScaler
19 import time
20
21 warnings.filterwarnings("ignore")
22
23 CSV_FOLDER = "All_csv"
24 RANDOM_STATE = 42

```



```

25 TEST_SIZE = 0.2
26 CV_FOLDS = 5
27
28
29 #
-----
30 # 1. Load and unify all datasets
31 #
-----
32
33 def load_datasets(folder: str) -> pd.DataFrame:
34     dfs = []
35
36     # --- Files with proper headers and last column as label ---
37     named_label = {
38         "dataset (2).csv": "label",
39         "dataset_3_3.csv": "contaminacion",
40         "dataset_clasificacion.csv": "label",
41         "dataset_con_etiquetas.csv": "etiqueta",
42         "matriz_final.csv": "etiqueta_arroz",
43     }
44     for fname, label_col in named_label.items():
45         path = os.path.join(folder, fname)
46         df = pd.read_csv(path)
47         df = df.rename(columns={label_col: "label"})
48         # keep only 16384 pixel columns + label
49         df = df.iloc[:, :16384].assign(label=df["label"])
50         dfs.append(df)
51
52     # --- Semicolon-delimited file ---
53     df = pd.read_csv(os.path.join(folder, "dataset_imagenes.csv"), sep=";",
54                     )
55     df = df.rename(columns={"etiqueta": "label"})
56     df = df.iloc[:, :16384].assign(label=df["label"])
57     dfs.append(df)
58
59     # --- Files without header (first row is data) ---
60     no_header_files = [
61         "dataset (1).csv",
62         "dataset.csv",
63         "dataset_C26797.csv",
64         "dataset_daniel_valverde.csv",
65         "matriz_final (1).csv",
66     ]
67     for fname in no_header_files:
68         path = os.path.join(folder, fname)
69         df = pd.read_csv(path, header=None)
70         df.columns = [f"pixel_{i}" for i in range(df.shape[1] - 1)] +
71             ["label"]
72         dfs.append(df)

```

```

72     combined = pd.concat(dfs, ignore_index=True)
73
74     # Drop exact duplicates
75     before = len(combined)
76     combined = combined.drop_duplicates()
77     after = len(combined)
78     print(f"    Loaded {before} rows total, {before - after} duplicates
79           removed -> {after} unique samples")
80
81     return combined
82
83 #
84 -----
85
86 # 2. Evaluate a model with GridSearchCV and return a results dict
87 #
88 -----
89
90 def evaluate(name: str, pipeline: Pipeline, param_grid: dict,
91             X_train, X_test, y_train, y_test) -> dict:
92
93     cv = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state
94                           =RANDOM_STATE)
95     grid = GridSearchCV(pipeline, param_grid, cv=cv, scoring="f1",
96                          n_jobs=-1, refit=True)
97
98     t0 = time.time()
99     grid.fit(X_train, y_train)
100     elapsed = time.time() - t0
101
102     y_pred = grid.predict(X_test)
103
104     result = {
105         "Model": name,
106         "Best params": grid.best_params_,
107         "CV F1": round(grid.best_score_, 4),
108         "Accuracy": round(accuracy_score(y_test, y_pred), 4),
109         "Precision": round(precision_score(y_test, y_pred,
110                                           zero_division=0), 4),
111         "Recall": round(recall_score(y_test, y_pred,
112                                     zero_division=0), 4),
113         "F1 (test)": round(f1_score(y_test, y_pred, zero_division=0),
114                             4),
115         "Time (s)": round(elapsed, 1),
116         "Confusion": confusion_matrix(y_test, y_pred).tolist(),
117         "Report": classification_report(y_test, y_pred,
118                                       target_names=["Negativo(0)", "Positivo(1)"]),
119     }
120     return result
121
122
123

```

```

114 #
115 # 3. Main
116 #
117
118 def main():
119     print("=" * 60)
120     print("LOADING DATASETS")
121     print("=" * 60)
122     data = load_datasets(CSV_FOLDER)
123
124     X = data.iloc[:, :16384].values.astype(np.uint8)
125     y = data["label"].values.astype(np.uint8)
126     print(f"  Features: {X.shape[1]} | Samples: {X.shape[0]}")
127     print(f"  Class distribution -> 0: {int(np.sum(y==0))} 1: {int(np.sum(y==1))}")
128
129     # Train/test split (80/20, stratified)
130     X_train, X_test, y_train, y_test = train_test_split(
131         X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y
132     )
133     print(f"\n  Train: {len(X_train)}  Test: {len(X_test)}\n")
134
135     # VarianceThreshold removes pixels that never change across
136     # training samples
137     vt = VarianceThreshold(threshold=0.0)
138     X_train_vt = vt.fit_transform(X_train)
139     X_test_vt = vt.transform(X_test)
140     print(f"  Features after VarianceThreshold: {X_train_vt.shape[1]}")
141
142     print("\n" + "=" * 60)
143     print("TRAINING MODELS")
144     print("=" * 60)
145
146     results = []
147
148     # --- Decision Tree ---
149     print("\n[1/4] Decision Tree...")
150     pipe_dt = Pipeline([("clf", DecisionTreeClassifier(random_state=
151         RANDOM_STATE))])
152     grid_dt = {
153         "clf__criterion": ["gini", "entropy"],
154         "clf__max_depth": [3, 5, 10, 20, None],
155         "clf__min_samples_split": [2, 5, 10],
156     }
157     results.append(evaluate("Decision Tree", pipe_dt, grid_dt,
158                             X_train_vt, X_test_vt, y_train, y_test))

```

```

157 print(f"    Best: {results[-1]['Best params']} | Test F1: {results
158       [-1]['F1 (test)']}")
159
160 # --- Random Forest ---
161 print("\n[2/5] Random Forest...")
162 pipe_rf = Pipeline([("clf", RandomForestClassifier(random_state=
163     RANDOM_STATE, n_jobs=-1))])
164 grid_rf = {
165     "clf__n_estimators": [50, 100, 200],
166     "clf__max_depth":    [5, 10, 20, None],
167     "clf__criterion":    ["gini", "entropy"],
168 }
169 results.append(evaluate("Random Forest", pipe_rf, grid_rf,
170     X_train_vt, X_test_vt, y_train, y_test))
171 print(f"    Best: {results[-1]['Best params']} | Test F1: {results
172       [-1]['F1 (test)']}")
173
174 # --- Naive Bayes (Bernoulli, ideal for binary features) ---
175 print("\n[3/5] Bernoulli Naive Bayes...")
176 pipe_nb = Pipeline([("clf", BernoulliNB())])
177 grid_nb = {
178     "clf__alpha": [0.001, 0.01, 0.1, 0.5, 1.0, 2.0, 5.0],
179 }
180 results.append(evaluate("Naive Bayes (Bernoulli)", pipe_nb,
181     grid_nb,
182     X_train_vt, X_test_vt, y_train, y_test))
183 print(f"    Best: {results[-1]['Best params']} | Test F1: {results
184       [-1]['F1 (test)']}")
185
186 # --- KNN ---
187 print("\n[4/5] K-Nearest Neighbors...")
188 pipe_knn = Pipeline([
189     ("scaler", StandardScaler()),
190     ("clf", KNeighborsClassifier()),
191 ])
192 grid_knn = {
193     "clf__n_neighbors": [3, 5, 7, 11, 15],
194     "clf__weights":     ["uniform", "distance"],
195     "clf__metric":      ["euclidean", "manhattan"],
196 }
197 results.append(evaluate("KNN", pipe_knn, grid_knn,
198     X_train_vt, X_test_vt, y_train, y_test))
199 print(f"    Best: {results[-1]['Best params']} | Test F1: {results
200       [-1]['F1 (test)']}")
201
202 # --- SVM (Linear, efficient for high-dimensional binary data) ---
203 print("\n[5/5] SVM (LinearSVC)...")
204 pipe_svm = Pipeline([
205     ("scaler", StandardScaler()),
206     ("clf", LinearSVC(random_state=RANDOM_STATE, max_iter=5000)
207     ),
208 ])
209 grid_svm = {

```

```

203         "clf__C": [0.001, 0.01, 0.1, 1.0, 10.0],
204         "clf__loss": ["hinge", "squared_hinge"],
205     }
206     results.append(evaluate("SVM (Linear)", pipe_svm, grid_svm,
207                             X_train_vt, X_test_vt, y_train, y_test))
208     print(f"    Best: {results[-1]['Best params']} | Test F1: {results
209           [-1]['F1 (test)']}")
210
211     #
212     -----
213
214     # 4. Summary table
215     #
216     -----
217
218     print("\n" + "=" * 60)
219     print("RESULTS SUMMARY")
220     print("=" * 60)
221
222     summary = pd.DataFrame([
223         {
224             "Model": r["Model"],
225             "Accuracy": r["Accuracy"],
226             "Precision": r["Precision"],
227             "Recall": r["Recall"],
228             "F1 (test)": r["F1 (test)"],
229             "CV F1": r["CV F1"],
230             "Time (s)": r["Time (s)"],
231         } for r in results])
232
233     summary = summary.sort_values("F1 (test)", ascending=False).
234         reset_index(drop=True)
235     print(summary.to_string(index=False))
236
237     best = summary.iloc[0]["Model"]
238     print(f"\nBest model: {best}")
239
240     # Detailed reports
241     print("\n" + "=" * 60)
242     print("DETAILED CLASSIFICATION REPORTS")
243     print("=" * 60)
244     for r in results:
245         print(f"\n--- {r['Model']} ---")
246         print(f"Best hyperparameters: {r['Best params']}")
247         print(f"Confusion matrix:\n{np.array(r['Confusion'])}")
248         print(r["Report"])
249
250     # Save summary CSV
251     summary.to_csv("classification_results.csv", index=False)
252     print("Summary saved to 'classification_results.csv'")
253
254     if __name__ == "__main__":
255         main()

```

E. Código Python para la exportación del modelo

```
1 import os
2 import numpy as np
3 import pandas as pd
4 import joblib
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.feature_selection import VarianceThreshold
7 from sklearn.pipeline import Pipeline
8
9 # Best hyperparameters found during GridSearchCV
10 BEST_PARAMS = {
11     "criterion": "entropy",
12     "max_depth": 10,
13     "n_estimators": 200,
14     "random_state": 42,
15     "n_jobs": -1,
16 }
17
18 CSV_FOLDER = "All_csv"
19 OUTPUT_FILE = "B83417_Jacob_Gonzalez.joblib"
20
21
22 def load_datasets(folder: str) -> pd.DataFrame:
23     dfs = []
24
25     named_label = {
26         "dataset (2).csv": "label",
27         "dataset_3_3.csv": "contaminacion",
28         "dataset_clasificacion.csv": "label",
29         "dataset_con_etiquetas.csv": "etiqueta",
30         "matriz_final.csv": "etiqueta_arroz",
31     }
32     for fname, label_col in named_label.items():
33         df = pd.read_csv(os.path.join(folder, fname))
34         df = df.rename(columns={label_col: "label"})
35         df = df.iloc[:, :16384].assign(label=df["label"])
36         dfs.append(df)
37
38     df = pd.read_csv(os.path.join(folder, "dataset_imagenes.csv"), sep=";")
39     df = df.rename(columns={"etiqueta": "label"})
40     df = df.iloc[:, :16384].assign(label=df["label"])
41     dfs.append(df)
42
43     for fname in ["dataset (1).csv", "dataset.csv", "dataset_C26797.csv",
44                 "dataset_daniel_valverde.csv", "matriz_final (1).csv
45                 "]:
46         df = pd.read_csv(os.path.join(folder, fname), header=None)
47         df.columns = [f"pixel_{i}" for i in range(df.shape[1] - 1)] +
48             ["label"]
49         dfs.append(df)
```

```

48
49     combined = pd.concat(dfs, ignore_index=True).drop_duplicates()
50     return combined
51
52
53 def main():
54     print("Loading full dataset...")
55     data = load_datasets(CSV_FOLDER)
56     X = data.iloc[:, :16384].values.astype(np.uint8)
57     y = data["label"].values.astype(np.uint8)
58     print(f"    Samples: {len(X)}    |    Class 0: {int(np.sum(y==0))}
          Class 1: {int(np.sum(y==1))}")
59
60     print(f"Training Random Forest on full dataset with best params: {
          BEST_PARAMS}")
61     model = Pipeline([
62         ("vt", VarianceThreshold(threshold=0.0)),
63         ("clf", RandomForestClassifier(**BEST_PARAMS)),
64     ])
65     model.fit(X, y)
66
67     joblib.dump(model, OUTPUT_FILE)
68     size_kb = os.path.getsize(OUTPUT_FILE) / 1024
69     print(f"Model saved as '{OUTPUT_FILE}' ({size_kb:.1f} KB)")
70     print("Done.")
71
72
73 if __name__ == "__main__":
74     main()

```

Referencias

- [1] SciPy Developers, *scipy.signal.windows.lanczos* — *SciPy v1.13.0 Manual*, Accedido el 22 de mayo de 2024, 2024. dirección: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.windows.lanczos.html>.